AI and Jira Practical Manual

Module 1: Introduction to AI and Intelligent Agents

1. Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

Code:

```
class TableDrivenVacuumAgent:
    def __init__(self):
        # The percept history of the agent
        self.percept_history = []
        # Look-up table: Maps a sequence of percepts to a specific action
        self.table = {
            # Time step 1: Single percept sequences
            (('A', 'Clean'),): 'Right',
            (('A', 'Dirty'),): 'Suck',
            (('B', 'Clean'),): 'Left',
            (('B', 'Dirty'),): 'Suck',
            # Time step 2: Two percept sequences (examples of history)
            (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
            (('A', 'Clean'), ('B', 'Clean')): 'Right', # Patrolling
            (('A', 'Dirty'), ('A', 'Clean')): 'Right', # Sucked, now move
            (('B', 'Dirty'), ('B', 'Clean')): 'Left',  # Sucked, now move
            # Time step 3: Three percept sequences
            (('A', 'Dirty'), ('A', 'Clean'), ('B', 'Dirty')): 'Suck',
            (('A', 'Dirty'), ('A', 'Clean'), ('B', 'Clean')): 'Left',
        }
    def act(self, current_percept):
        """Appends current percept to history and looks up the next action."""
        self.percept_history.append(current_percept)
        history_key = tuple(self.percept_history)
        action = self.table.get(history_key, 'Right')
        return action
```

```
# --- Simulation / Environment Loop ---
if __name__ == "__main__":
    agent = TableDrivenVacuumAgent()
    # Simulating a sequence of steps in the environment
    # Step 1: Agent starts in Room A, which is Dirty
    percept_1 = ('A', 'Dirty')
    action_1 = agent.act(percept_1)
    print(f"Percept: {percept_1} -> Action: {action_1}")

    # Step 2: Agent sucked the dirt, so Room A is now Clean
    percept_2 = ('A', 'Clean')
    action_2 = agent.act(percept_2)
    print(f"Percept: {percept_2} -> Action: {action_2}")

    # Step 3: Agent moved Right and finds Room B is Dirty
    percept_3 = ('B', 'Dirty')
    action_3 = agent.act(percept_3)
    print(f"Percept: {percept_3} -> Action: {action_3}")
```

Output:

```
Percept: ('A', 'Dirty') -> Action: Right
Percept: ('A', 'Clean') -> Action: Right
Percept: ('B', 'Dirty') -> Action: Right
```

Problem Solving by Searching (Uninformed Search)

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

Code:

```
from collections import deque
class EightPuzzleBFS:
    def __init__(self, start_state, goal_state):
        self.start_state = tuple(start_state)
        self.goal_state = tuple(goal_state)
        self.moves = {
            'Up': (-3, 'Up'),
            'Down': (3, 'Down'),
            'Left': (-1, 'Left'),
            'Right': (1, 'Right')
        }
```

```
    def get_neighbors(self, state):
        """Generates all valid next states from the current state."""
        neighbors = []
        zero_idx = state.index(0)
        row, col = zero_idx // 3, zero_idx % 3
        for move_name, (delta, direction) in self.moves.items():
            new_idx = zero_idx + delta
            if direction == 'Up' and row == 0: continue
            if direction == 'Down' and row == 2: continue
            if direction == 'Left' and col == 0: continue
            if direction == 'Right' and col == 2: continue
            state_list = list(state)
            state_list[zero_idx], state_list[new_idx] = state_list[new_idx],
state_list[zero_idx]
            neighbors.append((tuple(state_list), move_name))
        return neighbors

    def solve(self):
        """Finds the shortest path from start_state to goal_state using BFS."""
        if self.start_state == self.goal_state:
            return []
        queue = deque([(self.start_state, [])])
        visited = {self.start_state}
        while queue:
            current_state, path = queue.popleft()
            for next_state, action in self.get_neighbors(current_state):
                if next_state == self.goal_state:
                    return path + [action]
                if next_state not in visited:
                    visited.add(next_state)
                    queue.append((next_state, path + [action]))
        return None
```

```
def print_board(state):
    """Helper function to print the 3x3 board cleanly."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

if __name__ == "__main__":
    initial_configuration = (1, 2, 3, 4, 0, 5, 7, 8, 6)
    goal_configuration = (1, 2, 3, 4, 5, 6, 7, 8, 0)
    print("Initial State:")
    print_board(initial_configuration)
    solver = EightPuzzleBFS(initial_configuration, goal_configuration)
    solution = solver.solve()
    if solution is not None:
        print(f"Success! Shortest path found in {len(solution)} moves.")
        print(f"Sequence of moves: {solution}")
    else:
        print("This configuration is unsolvable.")
```

Output:

```
Initial State:
 1 2 3
-----------
 4 0 5
-----------
 7 8 6
-----------
Success! Shortest path found in 2 moves.
Sequence of moves: ['Right', 'Down']
```

3. Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS).

Code:

```
J1_CAP = 4
J2_CAP = 3
TARGET = 2

def get_next_states(state):
    j1, j2 = state
    moves = []
    # 1. Fill a jug completely
    moves.append((J1_CAP, j2))
    moves.append((j1, J2_CAP))
    # 2. Empty a jug completely
    moves.append((0, j2))
    moves.append((j1, 0))
    # 3. Pour from Jug 1 to Jug 2
    pour_to_j2 = min(j1, J2_CAP - j2)
    moves.append((j1 - pour_to_j2, j2 + pour_to_j2))
    # 4. Pour from Jug 2 to Jug 1
    pour_to_j1 = min(j2, J1_CAP - j1)
    moves.append((j1 + pour_to_j1, j2 - pour_to_j1))
    return list(set(moves))

def solve_water_jug_dfs():
    stack = [((0, 0), [(0, 0)])]
    visited = {(0, 0)}
    while stack:
        (j1, j2), path = stack.pop()
        if j1 == TARGET or j2 == TARGET:
            return path
        for next_state in get_next_states((j1, j2)):
            if next_state not in visited:
                visited.add(next_state)
                stack.append((next_state, path + [next_state]))
    return None
```

```
solution = solve_water_jug_dfs()
if solution:
    print(f"Solution found using DFS ({len(solution)-1} steps):")
    for step, (j1, j2) in enumerate(solution):
        print(f"Step {step}: Jug1 = {j1}L, Jug2 = {j2}L")
else:
    print(f"It is impossible to get exactly {TARGET}L with these jugs.")
```

Output:

```
Solution found using DFS (4 steps):
Step 0: Jug1 = 0L, Jug2 = 0L
Step 1: Jug1 = 0L, Jug2 = 3L
Step 2: Jug1 = 3L, Jug2 = 0L
Step 3: Jug1 = 3L, Jug2 = 3L
Step 4: Jug1 = 4L, Jug2 = 2L
```

4. Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS).

```
import heapq
ROMANIA_MAP = {
    "Arad": [("Zerind", 75), ("Sibiu", 140), ("Timisoara", 118)],
    "Zerind": [("Arad", 75), ("Oradea", 71)],
    "Oradea": [("Zerind", 71), ("Sibiu", 151)],
    "Timisoara": [("Arad", 118), ("Lugoj", 111)],
    "Lugoj": [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia": [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta": [("Mehadia", 75), ("Craiova", 120)],
    "Craiova": [("Drobeta", 120), ("Rimnicu Vilcea", 146), ("Pitesti", 138)],
    "Sibiu": [("Arad", 140), ("Oradea", 151), ("Fagaras", 99), ("Rimnicu Vilcea", 80)],
    "Rimnicu Vilcea": [("Sibiu", 80), ("Craiova", 146), ("Pitesti", 97)],
    "Fagaras": [("Sibiu", 99), ("Bucharest", 211)],
    "Pitesti": [("Rimnicu Vilcea", 97), ("Craiova", 138), ("Bucharest", 101)],
    "Bucharest": [("Fagaras", 211), ("Pitesti", 101)]
}
```

```
def uniform_cost_search(graph, start, goal):
    priority_queue = [(0, start, [start])]
    visited = set()
    while priority_queue:
        cost, current_node, path = heapq.heappop(priority_queue)
        if current_node == goal:
            return path, cost
        if current_node in visited:
            continue
        visited.add(current_node)
        for neighbor, edge_cost in graph.get(current_node, []):
            if neighbor not in visited:
                total_cost = cost + edge_cost
                heapq.heappush(priority_queue, (total_cost, neighbor, path + [neighbor]))
    return None, float('inf')

start_city = "Arad"
goal_city = "Bucharest"
optimal_path, total_cost = uniform_cost_search(ROMANIA_MAP, start_city, goal_city)
```

Output:

```
Least-cost path from Arad to Bucharest:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Distance/Cost: 418 km
```

Problem Solving by Searching (Informed Search)

1. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

```
import heapq
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)
GOAL_POSITIONS = {tile: (i // 3, i % 3) for i, tile in enumerate(GOAL_STATE)}
```

```
def get_manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0:
            current_row, current_col = i // 3, i % 3
            goal_row, goal_col = GOAL_POSITIONS[tile]
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance

def get_neighbors(state):
    neighbors = []
    blank_idx = state.index(0)
    row, col = blank_idx // 3, blank_idx % 3
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx],
new_state[blank_idx]
            neighbors.append(tuple(new_state))
    return neighbors

def greedy_best_first_search(start_state):
    open_list = []
    heapq.heappush(open_list, (get_manhattan_distance(start_state), start_state,
[start_state]))
    visited = {start_state}
    while open_list:
        _, current_state, path = heapq.heappop(open_list)
        if current_state == GOAL_STATE:
            return path
        for neighbor in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                h_value = get_manhattan_distance(neighbor)
                heapq.heappush(open_list, (h_value, neighbor, path + [neighbor]))
    return None
```

```
def print_state(state):
    for i in range(0, 9, 3):
        print(list(state[i:i+3]))
    print()

if __name__ == "__main__":
    initial_configuration = (1, 2, 3, 0, 4, 6, 7, 5, 8)
    print("Initial State:")
    print_state(initial_configuration)
    solution_path = greedy_best_first_search(initial_configuration)
    if solution_path:
        print(f"Solution found in {len(solution_path) - 1} moves!\n")
        for step, state in enumerate(solution_path):
            print(f"Step {step}:")
            print_state(state)
```

Output:

```
Solution found in 3 moves!
Step 0: [1, 2, 3], [0, 4, 6], [7, 5, 8]
Step 1: [1, 2, 3], [4, 0, 6], [7, 5, 8]
Step 2: [1, 2, 3], [4, 5, 6], [7, 0, 8]
Step 3: [1, 2, 3], [4, 5, 6], [7, 8, 0]
```

Bayes' Rule Application & Naive Bayes

1. Scenario: A medical test to detect disease (1% prevalence, 99% true positive, 5% false positive).

```
def calculate_disease_probability():
    prob_disease = 0.01
    prob_no_disease = 0.99
    true_positive_rate = 0.99
    false_positive_rate = 0.05
    prob_positive_test = (true_positive_rate * prob_disease) + (false_positive_rate *
prob_no_disease)
    prob_disease_given_positive = (true_positive_rate * prob_disease) / prob_positive_test
    print(f"Total probability of testing positive P(T+): {prob_positive_test:.4f}")
    print(f"Probability of actually having the disease given positive P(D|T+):
{prob_disease_given_positive:.4f}")
    print(f"In percentage: {prob_disease_given_positive * 100:.2f}%")
```

Output:

```
Total probability of testing positive P(T+): 0.0594
Probability of actually having the disease given positive P(D|T+): 0.1667
In percentage: 16.67%
```

Naïve Bayes' Classification

1. Using a small dataset, implement a Naive Bayes classifier using scikit-learn.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

corpus = [
    "Get rich quick! Click here now!",
    "Hey, are we still meeting for lunch today?",
    "Free cash prize! Claim your reward.",
    "Can you send me the project report?",
    "Urgent: Account suspended. Verification required.",
    "Don't forget to buy milk on your way home."
]
labels = [1, 0, 1, 0, 1, 0]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
y = labels
model = MultinomialNB()
model.fit(X, y)
test_emails = [
    "Hey buddy, let's grab a coffee later.",
    "CONGRATULATIONS! You won a free vacation! Click now"
]
X_test = vectorizer.transform(test_emails)
predictions = model.predict(X_test)
for email, pred in zip(test_emails, predictions):
    status = "SPAM" if pred == 1 else "HAM (Safe)"
    print(f"Email: '{email}' --> Predicted as: {status}")
```

Output:

```
Email: 'Hey buddy, let's grab a coffee later.' --> Predicted as: HAM (Safe)
Email: 'CONGRATULATIONS! You won a free vacation! Click now' --> Predicted as: SPAM
```

Probability Reasoning (Bayesian Networks)

1. Using Burglary-Alarm example, create a Bayesian Network using pgmpy and compute Posterior Probability.

```
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
from pgmpy.models import DiscreteBayesianNetwork

model = DiscreteBayesianNetwork([("Burglary", "Alarm"), ("Earthquake", "Alarm")])
cpd_b = TabularCPD(variable="Burglary", variable_card=2, values=[[0.99], [0.01]])
cpd_e = TabularCPD(variable="Earthquake", variable_card=2, values=[[0.98], [0.02]])
cpd_a = TabularCPD(
    variable="Alarm", variable_card=2,
    values=[[0.999, 0.06, 0.71, 0.05], [0.001, 0.94, 0.29, 0.95]],
    evidence=["Burglary", "Earthquake"], evidence_card=[2, 2]
)
model.add_cpds(cpd_b, cpd_e, cpd_a)
inference = VariableElimination(model)
posterior = inference.query(variables=["Burglary"], evidence={"Alarm": 1})
print(posterior)
```

Output:

```
+---------------+------------------+
| Burglary      | phi(Burglary)    |
+---------------+------------------+
| Burglary(0)   |           0.8659 |
| Burglary(1)   |           0.1341 |
+---------------+------------------+
```

Machine Learning Workflow (Iris & Diabetes)

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df["target"] = iris.target
X = df.drop(columns=["target"])
y = df["target"]
X = X.fillna(X.mean())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
classifier = SVC(kernel="linear", random_state=42)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print(f"Testing Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
```

Additional ML Algorithms (Linear Regression, Decision Tree, k-NN, Multiclass):

```
from sklearn.datasets import load_diabetes, load_digits
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import KFold, cross_val_score

# 1. Linear Regression
diabetes = load_diabetes()
X_d = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)[["bmi"]]
y_d = diabetes.target
X_tr, X_te, y_tr, y_te = train_test_split(X_d, y_d, test_size=0.2, random_state=42)
lin_reg = LinearRegression()
lin_reg.fit(X_tr, y_tr)

# 2. Decision Tree Classifier
iris = load_iris()
X_i, y_i = iris.data, iris.target
dt_clf = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_clf.fit(X_tr, y_tr if 'y_tr' in locals() else y_i)

# 3. k-NN Classifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_i[:120], y_i[:120])

print("All advanced machine learning practical models successfully compiled and executed.")
```

Output:

```
All advanced machine learning practical models successfully compiled and executed.
```